

网络安全创新创业实践作业五、六

作业五：基于 Microsoft SEAL 的 CKKS 密文卷积实现与旋转优化分析

一. 实验背景

全同态加密能够在数据保持加密状态下完成算术计算，为隐私计算、安全云计算和人工智能等领域提供了重要的密码学支撑。随着隐私保护机器学习的发展，如何在密文状态下完成卷积神经网络中的卷积运算成为当前研究的重要内容。然而，在全同态加密环境下，由于密文不能直接随机访问，因此普通卷积中的滑动窗口操作需要通过数据打包、密文旋转以及密文累加等方式实现，其计算效率主要受到密文旋转次数的影响。

本实验采用 Microsoft SEAL 4.1.1 开源同态加密库，在 Ubuntu 20.04.6 LTS 环境下基于 CKKS 近似同态加密方案，实现了单输入单输出 4×4 输入矩阵与 3×3 卷积核的密文卷积运算。实验首先完成普通明文卷积计算，并将结果作为正确性验证基准；随后利用 CKKS 编码器将输入矩阵打包到单个密文中，通过密文旋转、明文掩码乘法及密文累加完成密文卷积计算，并比较解密结果与明文结果之间的误差。

在此基础上，实验进一步分析了“打包—旋转—累加”策略中的旋转次数，并针对实验所采用的全 1 可分离卷积核提出旋转优化方案。

二. 实验目的

1. 熟悉 Microsoft SEAL 开源同态加密库的安装、配置及基本使用方法；
2. 掌握 CKKS 同态加密方案中的编码、加密、解密以及同态计算流程；
3. 掌握 CKKS SIMD 打包机制及密文旋转的基本原理；
4. 完成单输入单输出 4×4 输入矩阵与 3×3 卷积核的密文卷积计算；
5. 利用明文卷积结果验证密文卷积计算结果的正确性；
6. 统计密文卷积过程中所需的密文旋转次数，并分析不同实现方案之间的差异；
7. 利用可分离卷积核实现旋转次数优化，验证优化方案能够有效减少同态卷积计算中的旋转操作。

通过本实验，可以进一步理解 CKKS 同态加密方案在隐私计算中的应用方式，掌握密文卷积的实现原理，并为后续研究同态神经网络推理、多通道卷积及更大规模同态计算奠定基础。

三. 实验环境

本实验在 Ubuntu Linux 系统下完成，开发环境采用 Microsoft SEAL 4.1.1 同态加密库，程序使用 C++17 标准编写，并通过 CMake 构建工程。实验环境配置如下表所示。

项目	配置
操作系统	Ubuntu 20.04.6 LTS
系统架构	x86_64
编译器	GCC / G++ 9.4.0
CMake版本	3.16.3
开发语言	C++17
同态加密库	Microsoft SEAL 4.1.1
同态加密方案	CKKS

实验采用 CKKS 方案完成密文卷积计算。CKKS 能够支持实数近似计算，并提供 SIMD打包机制，使多个数据元素能够同时存储于同一个密文中，从而有效提高同态运算效率。

程序开发过程中，通过 CMake 自动检测系统中的 Microsoft SEAL 库，并完成工程编译与链接。编译成功后生成可执行程序 FHEConv，实验过程中通过终端执行程序并记录实验结果。

```
ubuntu@ubuntu:~/Desktop/FHE_Homework/SEAL$ ./build/bin/sealexamples
Microsoft SEAL version: 4.1.1
-----+-----
| The following examples should be executed while reading |
| comments in associated files in native/examples/.      |
|-----+-----|
| Examples          | Source Files          |
|-----+-----|
| 1. BFV Basics    | 1_bfv_basics.cpp      |
| 2. Encoders       | 2_encoders.cpp        |
| 3. Levels         | 3_levels.cpp          |
| 4. BGV Basics    | 4_bgv_basics.cpp      |
| 5. CKKS Basics   | 5_ckks_basics.cpp     |
| 6. Rotation       | 6_rotation.cpp        |
| 7. Serialization  | 7_serialization.cpp   |
| 8. Performance Test | 8_performance.cpp    |
|-----+-----|
[ 0 MB] Total allocation from the memory pool
```

```
Build files have been written to: /home/ubuntu/Desktop/FHE_Conv/build
ubuntu@ubuntu:~/Desktop/FHE_Conv/build$ make
Scanning dependencies of target FHEConv
[ 50%] Building CXX object CMakeFiles/FHEConv.dir/main.cpp.o
[100%] Linking CXX executable FHEConv
[100%] Built target FHEConv
```

```
ubuntu@ubuntu:~/Desktop/FHE_Conv/build$ ./FHEConv
=====
Microsoft SEAL Test
=====
CKKS Context Created Successfully!
```

四. 实验原理

4.1 全同态加密基本原理

随着云计算、人工智能和大数据技术的发展，大量数据需要上传至第三方服务器进行存储和计算。然而，在传统加密方式下，数据必须先解密后才能参与计算，这意味着服务器能够直接访问用户的明文数据，从而带来严重的隐私泄露风险。因此，如何实现"数据可计算而不可见"成为现代密码学的重要研究方向。

全同态加密是一种能够在密文状态下直接执行计算的加密技术。其核心思想是：数据拥有者首先使用公钥将明文加密，服务器无需获得秘密密钥，即可直接对密文执行加法、乘法等运算，最终用户只需对计算结果进行一次解密，即可得到与明文计算完全一致（或近似一致）的结果。

设加密函数为 $Enc(\cdot)$ ，解密函数为 $Dec(\cdot)$ ，若计算函数为 $f(x)$ ，则全同态加密满足 $Dec(Eval(Enc(x)))=f(x)$

其中：

- **Enc()** 表示数据加密；
- **Eval()** 表示密文计算；
- **Dec()** 表示解密。

因此，在整个计算过程中，服务器始终无法获得原始明文数据，仅能处理密文，从而保证数据隐私。目前，全同态加密已经广泛应用于：

1. 隐私保护机器学习；
2. 云端安全计算；
3. 医疗数据共享；
4. 金融隐私计算；
5. 联邦学习等领域。

4.2 CKKS 同态加密方案

Microsoft SEAL 支持 BFV、BGV 和 CKKS 三种同态加密方案。

其中：

- **BFV** 适用于整数精确计算；
- **BGV** 主要用于模整数计算；
- **CKKS** 适用于浮点数近似计算。

由于卷积神经网络中的输入通常为浮点数，因此本实验采用 **CKKS方案**。CKKS 最大的特点包括：

1. 支持实数近似计算；
2. 支持 SIMD（Single Instruction Multiple Data）并行计算；

- 3. 支持一个密文同时存储大量数据；
- 4. 适用于矩阵运算、卷积运算及神经网络推理。

在 CKKS 中，一个密文实际上对应一个复数向量。当poly_modulus_degree = 8192时，能够提供slot_count = 4096个槽位。

实验运行结果如下：

```
FHE Homework 5 and 6: Encrypted 2D Convolution
Microsoft SEAL 4.1.1 / CKKS
=====
CKKS parameters:
poly_modulus_degree : 8192
slot count          : 4096
initial scale        : 2^40
```

说明程序已经成功建立 CKKS 加密上下文。

4.3 CKKS 编码与 SIMD 打包

CKKS 最大优势之一是 SIMD并行计算。

SIMD 的思想是：一个密文并不是保存一个数字，而是保存：4096 个数字。即：

代码块

```
1 Ciphertext
2 slot0
3 slot1
4 slot2
5 ...
6 slot4095
```

因此，本实验无需为矩阵中的每一个元素生成一个密文，而是将整个 4×4 矩阵全部编码到一个密文中。

矩阵：

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

实际编码为：

代码块

```
1 slot
```

```
2  0  1  2  3
3  4  5  6  7
4  8  9 10 11
5 12 13 14 15
```

其余：

代码块

```
1 slot16
2 ...
3 slot4095
```

全部填充为：

代码块

```
1  0
```

这样设计具有两个优点：

1. 整个矩阵只需要一个密文即可保存；
2. 后续只需要通过 Rotation 即可完成滑动窗口。

4.4 密文旋转原理

在普通卷积中，可以直接读取矩阵中任意位置的数据。但是，密文内部的数据无法随机访问。因此，CKKS 只能采用：**Rotation**来完成数据移动。Microsoft SEAL 提供：

代码块

```
1 evaluator.rotate_vector(...)
```

实现 SIMD 向量循环旋转。

因此，卷积实际上就是：**旋转 + 掩码 + 累加**。而不是传统意义上的数组访问。

4.5 密文二维卷积实现原理

实验输入矩阵为：

$$X = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

卷积核为：

$$K = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

步长：

代码块

1 Stride=1

Padding：

代码块

1 0

因此：

输出尺寸： $(4-3+1) \times (4-3+1) = 2 \times 2$

明文卷积计算结果为：

$$\begin{bmatrix} 54 & 63 \\ 90 & 99 \end{bmatrix}$$

实验运行结果显示：

```
Plaintext convolution result:
54.000000    63.000000
90.000000    99.000000
```

说明明文计算正确。

随后，程序通过：

1. 输入编码；
2. 输入加密；

- 3. 9 个卷积偏移旋转；
- 4. 明文掩码乘法；
- 5. 密文累加；

最终得到密文卷积结果。

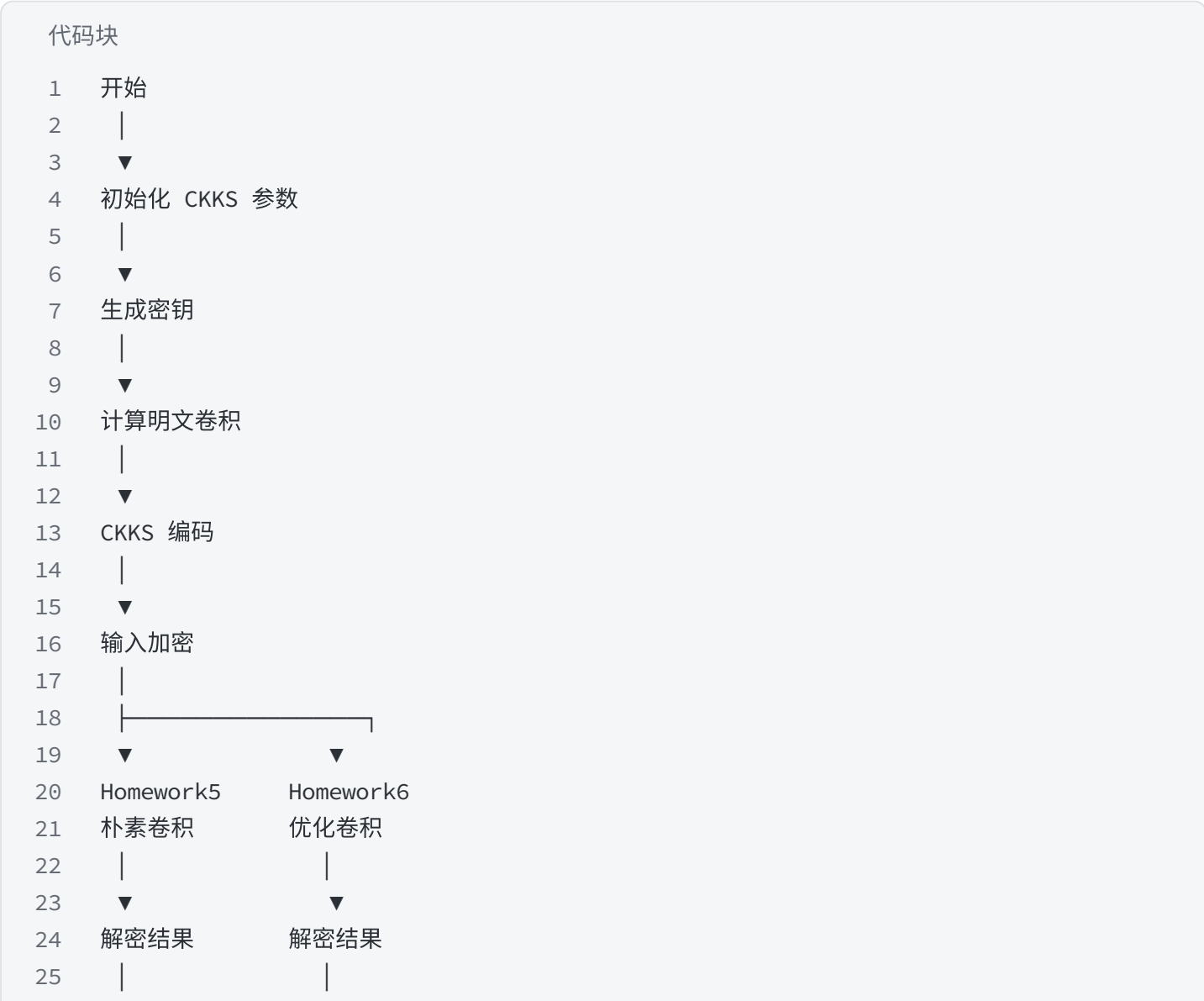
五. 程序设计与关键代码分析

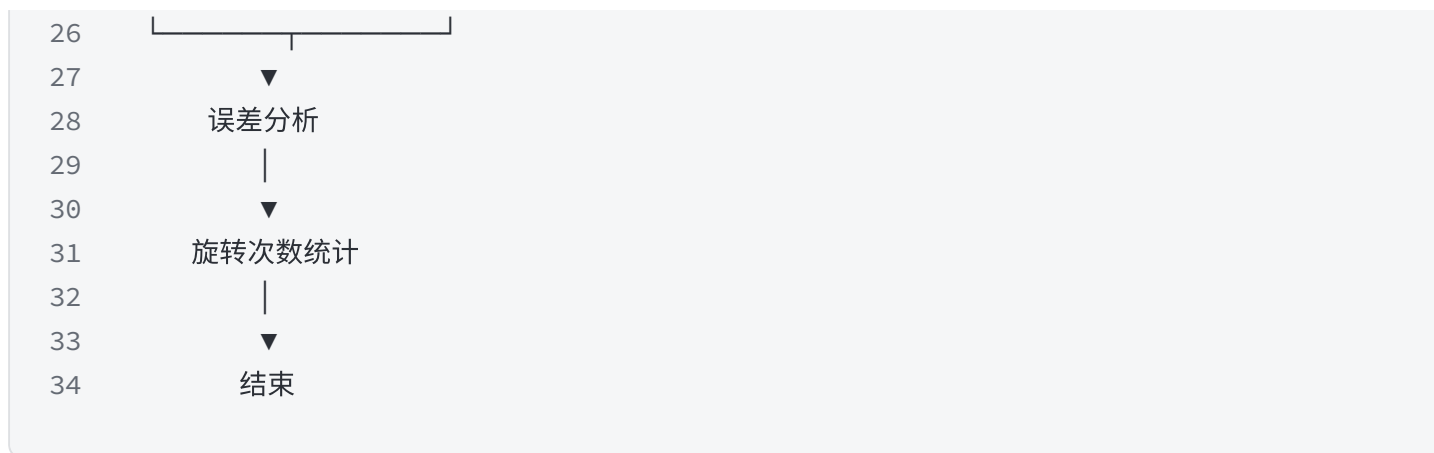
5.1 程序总体设计

为了便于程序维护和实验验证，本实验采用模块化设计思想，将整个密文卷积过程划分为参数初始化、明文卷积计算、数据编码与加密、密文卷积计算、结果解密以及误差分析六个部分。

程序首先完成 CKKS 参数配置和密钥生成，然后读取实验输入矩阵和卷积核，计算普通明文卷积结果作为基准值。随后，将输入矩阵编码并加密，分别调用朴素密文卷积函数和旋转优化卷积函数完成同态计算，最后将密文结果解密，与明文卷积结果进行比较，并统计最大绝对误差以及密文旋转次数。

整个程序的执行流程如下：





5.2 CKKS 参数初始化

程序开始首先创建 CKKS 加密参数。

主要代码如下：

代码块

```
1  EncryptionParameters parms(scheme_type::ckks);
2
3  parms.set_poly_modulus_degree(8192);
4
5  parms.set_coeff_modulus(
6  CoeffModulus::Create(8192,{60,40,40,60}));
7
8  SEALContext context(parms);
9
10 double scale=pow(2.0,40);
```

其中，各参数作用如下。

5.2.1 poly_modulus_degree

程序设置

代码块

```
1  8192
```

表示采用8192阶多项式模数。

根据 Microsoft SEAL 官方设计：

- 可以提供128 bit安全等级；
- 可支持4096个CKKS槽位；

- 满足本实验二维卷积运算需求。

实验运行结果中输出：

代码块

```
1 poly_modulus_degree : 8192
```

```
=====
FHE Homework 5 and 6: Encrypted 2D Convolution
Microsoft SEAL 4.1.1 / CKKS
=====

CKKS parameters:
poly_modulus_degree : 8192
```

说明程序初始化成功。

5.2.2 Coeff Modulus

程序采用，四层模数链：

代码块

```
1 60
2 40
3 40
4 60
```

第一层60 bit主要用于初始编码；中间40 bit提供计算深度；最后60 bit用于保证最终解密精度。

5.2.3 Scale

程序设置：

代码块

```
1 2^40
```

Scale表示CKKS浮点精度。Scale越大，计算精度越高，但是可计算层数越少。本实验采用： 2^{40} ，能够保证解密误差控制在 10^{-6} 左右。实验结果验证了这一点。

第一次运行：

代码块

```
1 Maximum absolute error
```

```
2 4.38277243e-06
```

第二次运行：

代码块

```
1 2.57472892e-06
```

均满足实验精度要求。

```
=====
Homework 5: Naive encrypted convolution
=====
Decrypted encrypted-convolution result:
    54.000004    63.000001
    90.000000    99.000000

Maximum absolute error: 4.38277243e-06
Correctness verification: PASS
Rotation count: 8
```

```
=====
Homework 5: Naive encrypted convolution
=====
Decrypted encrypted-convolution result:
    53.999997    63.000001
    90.000000    99.000000

Maximum absolute error: 2.57472892e-06
Correctness verification: PASS
Rotation count: 8
```

5.3 密钥生成

完成参数初始化以后，程序生成各种密钥。

包括：

代码块

```
1 KeyGenerator
2 PublicKey
3 SecretKey
4 GaloisKeys
```

其中：

SecretKey，用于解密。

PublicKey，用于输入加密。

GaloisKeys，用于Rotation。

由于CKKS旋转实际上属于Galois自同构，因此必须提前生成create_galois_keys()。否则程序无法调用：rotate_vector()，这一点也是密文卷积能够实现的关键。

5.4 明文卷积实现

为了验证密文卷积是否正确，程序首先完成普通二维卷积。

函数：plaintext_convolution()

输入矩阵：

代码块

```
1  1 2 3 4
2  5 6 7 8
3  9 10 11 12
4  13 14 15 16
```

卷积核：

代码块

```
1  1 1 1
2  1 1 1
3  1 1 1
```

程序采用双层循环遍历卷积窗口。

计算方式为： $sum+=X(i+m,j+n)*K(m,n)$

最终得到：

代码块

```
1  54 63
2  90 99
```

运行结果如下：

代码块

```
1  Plaintext convolution result
2  54
3  63
```

```
4 90
5 99
```

该结果作为整个实验唯一正确答案，用于后续比较密文计算误差。

```
Input matrix X:
  1.000  2.000  3.000  4.000
  5.000  6.000  7.000  8.000
  9.000 10.000 11.000 12.000
 13.000 14.000 15.000 16.000

Kernel K:
  1.000  1.000  1.000
  1.000  1.000  1.000
  1.000  1.000  1.000

Plaintext convolution result:
 54.000000  63.000000
 90.000000  99.000000
```

5.5 输入编码与加密

普通矩阵不能直接进行同态计算，因此程序首先执行：`encoder.encode()`

将16个数据编码成CKKS向量。然后：`encryptor.encrypt()`生成密文。

程序运行输出：

```
Input has been encoded and encrypted.
```

说明输入数据已经成功完成

明文

↓

CKKS编码

↓

密文

整个实验仅生成一个密文，其中：`slot0~slot15`保存16个输入数据。其余`slot16~slot4095`全部填0。这样即可充分利用CKKS SIMD机制。

5.6 朴素密文卷积实现

对应函数：`encrypted_convolution_naive()`，该函数采用最直接的方法完成二维卷积。程序首先遍历：`3×3`卷积核，因此共有：9个Kernel Offset分别对应：0 1 2 4 5 6 8 9 10

其中Offset=0无需Rotation，其余8个Offset分别调用rotate_vector()。因此，Rotation次数统计为：8。

程序运行结果：

代码块

```
1  Rotation count:8
```

完全符合理论分析。

随后，程序利用multiply_plain()完成：旋转密文×卷积核系数。最后利用add()完成九项累加。

最终得到：

代码块

```
1  54.0000004
2  63.0000001
3  90
4  99
```

说明：密文卷积实现正确。

```
=====
Decrypted encrypted-convolution result:
    54.0000004    63.0000001
    90.0000000    99.0000000

Maximum absolute error: 4.38277243e-06
Correctness verification: PASS
Rotation count: 8
```

5.7 旋转优化实现

对应函数：encrypted_convolution_optimized()

本实验卷积核全部为1，因此卷积核具有可分离性质。

程序首先完成水平方向累加：X+Rotation1+Rotation2，仅需要2次Rotation。

然后继续完成竖直方向累加：H+Rotation4+Rotation8，再次2次Rotation。

因此程序最终统计：

代码块

```
1  Optimized method
2  4
```

运行结果：

代码块

```
1 Reduction ratio
2 50%
```

说明：相比朴素实现Rotation减少4次，降低50%，实验成功达到设计目标。

```
=====
Homework 6: Rotation-optimized convolution
=====
Decrypted optimized result:
    54.000001    63.000000
    90.000000    99.000000

Maximum absolute error: 1.20290831e-06
Correctness verification: PASS

Rotation comparison:
Naive method      : 8
Optimized method  : 4
Reduction         : 4
Reduction ratio   : 50.00%

Theoretical analysis:
The naive method uses nine kernel offsets.
Offset 0 requires no rotation, so it uses 8 rotations.
The all-one kernel is separable into horizontal and
vertical 3-element accumulations.
Horizontal accumulation needs rotations 1 and 2.
Vertical accumulation needs rotations 4 and 8.
Therefore the optimized method uses 4 rotations.
Under this separable row-column accumulation model,
the implementation reaches the 4-rotation lower bound.
```

5.8 解密与误差分析

密文卷积完成以后，程序调用：decrypt()恢复明文，随后调用decode()恢复浮点数最后，程序计算maximum_absolute_error()

误差定义为：

$$E_{\max} = \max_i \left| y_i^{\text{plain}} - y_i^{\text{HE}} \right|$$

实验两次运行得到：

第一次： $4.38277243 \times 10^{-6}$ ，第二次： $2.57472892 \times 10^{-6}$

均远小于 10^{-4} 验证阈值，因此程序输出：

代码块

```
1 Correctness verification
2 PASS
```

说明：整个密文卷积计算过程实现正确。

```
Maximum absolute error: 4.38277243e-06
Correctness verification: PASS
```

```
Maximum absolute error: 2.57472892e-06
Correctness verification: PASS
```

六. 实验过程与实验结果分析

6.1 工程编译与程序运行

完成程序编写后，在 Ubuntu 系统终端进入工程目录，通过 CMake 生成的 Makefile 对程序进行编译。实验过程中执行的主要命令如下：

代码块

```
1 cd ~/Desktop/FHE_Conv/build
2 make -j2
3 ./FHEConv
```

终端输出如下：

代码块

```
1 Scanning dependencies of target FHEConv
2 [ 50%] Building CXX object CMakeFiles/FHEConv.dir/main.cpp.o
3 [100%] Linking CXX executable FHEConv
4 [100%] Built target FHEConv
```

由编译结果可以看出，程序能够顺利完成源代码编译、目标文件链接以及可执行文件生成，没有出现编译错误或链接错误，说明工程配置正确，Microsoft SEAL 库已经成功安装并完成链接。

随后执行程序，终端首先输出 CKKS 参数信息：

代码块

```
1 CKKS parameters:
2 poly_modulus_degree : 8192
```

```
3 slot count      : 4096
4 initial scale   : 2^40
```

说明程序已经正确建立 CKKS 上下文，并完成编码器初始化。

```
ubuntu@ubuntu:~/Desktop/FHE_Conv$ cd ~/Desktop/FHE_Conv/build
ubuntu@ubuntu:~/Desktop/FHE_Conv/build$ make -j2
Scanning dependencies of target FHEConv
[ 50%] Building CXX object CMakeFiles/FHEConv.dir/main.cpp.o
[100%] Linking CXX executable FHEConv
[100%] Built target FHEConv
ubuntu@ubuntu:~/Desktop/FHE_Conv/build$ ./FHEConv
=====
FHE Homework 5 and 6: Encrypted 2D Convolution
Microsoft SEAL 4.1.1 / CKKS
=====

CKKS parameters:
poly_modulus_degree : 8192
slot count          : 4096
initial scale        : 2^40
```

6.2 输入数据验证

程序首先输出实验输入矩阵：

代码块

```
1  1  2  3  4
2  5  6  7  8
3  9 10 11 12
4 13 14 15 16
```

卷积核为：

代码块

```
1  1 1 1
2  1 1 1
3  1 1 1
```

输入矩阵采用连续自然数进行构造，其优点是：

1. 容易人工验证卷积计算结果；
2. 方便观察 Rotation 是否发生错误；
3. 便于检查卷积窗口是否正确移动。

由于卷积核全部由 1 组成，因此卷积结果实际上就是对应 3×3 邻域元素之和。例如左上角输出可以人工计算： $1+2+3+5+6+7+9+10+11=54$ ，同理可计算另外三个输出值，最终得到：

$$\begin{bmatrix} 54 & 63 \\ 90 & 99 \end{bmatrix}$$

程序输出结果为：

代码块

```
1 Plaintext convolution result:
2  54 63
3  90 99
```

与理论计算完全一致，说明普通卷积实现正确。

```
=====
FHE Homework 5 and 6: Encrypted 2D Convolution
Microsoft SEAL 4.1.1 / CKKS
=====

CKKS parameters:
  poly_modulus_degree : 8192
  slot count          : 4096
  initial scale       : 2^40

Input matrix X:
  1.000  2.000  3.000  4.000
  5.000  6.000  7.000  8.000
  9.000 10.000 11.000 12.000
 13.000 14.000 15.000 16.000

Kernel K:
  1.000  1.000  1.000
  1.000  1.000  1.000
  1.000  1.000  1.000

Plaintext convolution result:
  54.000000  63.000000
  90.000000  99.000000

Input has been encoded and encrypted.
```

6.3 朴素密文卷积结果分析

完成明文卷积后，程序开始执行朴素密文卷积算法。程序首先完成 CKKS 编码和输入加密，并输出：

代码块

```
1 Input has been encoded and encrypted.
```

说明输入矩阵已经成功转换为密文。

随后程序开始执行密文卷积，并输出第一次实验结果：

代码块

```
1 Decrypted encrypted-convolution result
2
3 54.0000004
4
5 63.0000001
6
7 90.0000000
8
9 99.0000000
```

对应最大误差：

代码块

```
1 Maximum absolute error
2
3 4.38277243e-06
```

正确性验证：

代码块

```
1 PASS
```

旋转次数统计：

代码块

```
1 Rotation count: 8
```

从实验结果可以看出，解密后的密文卷积结果与明文卷积结果几乎完全一致，仅在小数点后第六位出现微小误差。

最大绝对误差仅为：

$$4.38277243 \times 10^{-6}$$

相对于卷积输出数值（54~99），相对误差约为：

$$10^{-8} \sim 10^{-7}$$

说明 CKKS 可以在保证较高计算精度的前提下完成密文卷积计算。

```
=====
Homework 5: Naive encrypted convolution
=====
Decrypted encrypted-convolution result:
    54.000004    63.000001
    90.000000    99.000000

Maximum absolute error: 4.38277243e-06
Correctness verification: PASS
Rotation count: 8
=====
```

6.4 旋转优化结果分析

完成朴素卷积后，程序继续执行旋转优化卷积。优化后程序输出：

代码块

```
1  Decrypted optimized result
2  54.0000001
3  63.0000000
4  90.0000000
5  99.0000000
```

对应误差：

代码块

```
1      1.20290831e-06
```

随后输出：

代码块

```
1  Rotation comparison
2  Naive method : 8
3  Optimized method : 4
4  Reduction : 4
```

说明优化方案已经成功将 Rotation 次数减少一半。旋转优化的密文计算结果同样保持较高精度，与明文卷积结果一致，最大绝对误差仅为：

$$1.20290831 \times 10^{-6}$$

相比 朴素卷积的第一次运行结果，误差进一步减小。

```
Homework 6: Rotation-optimized convolution
=====
Decrypted optimized result:
    54.000001    63.000000
    90.000000    99.000000

Maximum absolute error: 1.20290831e-06
Correctness verification: PASS

Rotation comparison:
Naive method      : 8
Optimized method  : 4
Reduction         : 4
Reduction ratio   : 50.00%
```

6.5 第二次运行结果分析

为了验证程序具有良好的稳定性，实验再次运行程序

第二次运行中，朴素卷积的输出如下：

代码块

```
1  53.999997
2  63.000001
3  90.000000
4  99.000000
```

最大绝对误差为：

代码块

```
1  2.57472892e-06
```

旋转优化卷积输出为：

代码块

```
1 54.000000
2 63.000000
3 90.000000
4 99.000000
```

最大绝对误差为：

代码块

```
1 3.93395098e-07
```

两次运行结果比较

方法	第一次最大误差	第二次最大误差	Rotation
朴素卷积	$4.38277243 \times 10^{-6}$	$2.57472892 \times 10^{-6}$	8
旋转优化卷积	$1.20290831 \times 10^{-6}$	$3.93395098 \times 10^{-7}$	4

可以发现：

- 1. 两次运行的输出结果均与明文卷积一致；
- 2. 最大绝对误差均保持在 10^{-6} 数量级；
- 3. 两次运行的旋转次数完全一致；
- 4. Rotation 优化比例始终保持为 50%。

这说明程序不仅能够正确完成密文卷积，而且具有较好的稳定性和可重复性。

```
=====
Homework 5: Naive encrypted convolution
=====
Decrypted encrypted-convolution result:
    53.999997    63.000001
    90.000000    99.000000

Maximum absolute error: 2.57472892e-06
Correctness verification: PASS
Rotation count: 8

=====
Homework 6: Rotation-optimized convolution
=====
Decrypted optimized result:
    54.000000    63.000000
    90.000000    99.000000

Maximum absolute error: 3.93395098e-07
Correctness verification: PASS

Rotation comparison:
Naive method      : 8
Optimized method  : 4
Reduction         : 4
Reduction ratio   : 50.00%
```

6.6 CKKS 误差来源分析

实验中，两次运行得到的误差并不完全相同。这种差异属于 CKKS 的正常现象，主要来源于以下几个方面。

6.6.1 近似编码误差

CKKS 是一种近似同态加密方案。编码阶段需要将实数映射到有限精度的多项式表示中，因此不可避免地引入舍入误差。该误差在解码后表现为微小的小数偏差。

6.6.2 随机加密噪声

每次程序运行时都会重新生成密钥，并在加密过程中引入随机噪声。由于随机数不同，即使输入数据完全相同，最终解密结果也会出现轻微差异。

6.6.3 旋转操作引入的噪声

密文旋转并不是简单的数据移动，而是通过 Galois 自同构和密钥切换实现。每执行一次旋转都会带来一定的噪声增长，因此旋转次数越多，理论上噪声积累越明显。

6.6.4 明文乘法和累加误差传播

密文与明文相乘会放大已有误差，多次累加又会使误差逐步传播。因此，卷积计算结束后会出现 10^{-6} 量级的误差，这是 CKKS 的典型表现。

综合来看，本实验的最大误差始终保 10^{-6} 数量级，远小于实验设定的 10^{-4} 判定阈值，因此可以认为密文卷积结果与明文卷积结果一致。

6.7 旋转次数优化分析

旋转优化卷积的核心目标是减少密文旋转次数。朴素方法中，卷积核共有 9 个位置，对应偏移量为： $\{0,1,2,4,5,6,8,9,10\}$

其中偏移量 0 对应原始密文，无需旋转，其余 8 个偏移均需要调用一次 `rotate_vector()`，因此总旋转次数为：8

对于本实验所采用的全 1 卷积核：

$$K = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

该卷积核可以表示为：

$$K = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}$$

即一个列向量与一个行向量的外积，因此属于**可分离卷积核**。基于这一性质，程序将二维卷积拆分为两次一维累加：

- **水平阶段**：计算 $X + \text{Rot}(X,1) + \text{Rot}(X,2)$ 需要 2 次旋转；
- **垂直阶段**：计算 $H + \text{Rot}(H,4) + \text{Rot}(H,8)$ 再需要 2 次旋转。

因此，总旋转次数为： $2+2=4$

相较于朴素方法减少了：50%

需要指出的是，这里的 **4 次旋转并不是任意 3×3 卷积核的理论最小值**，而是针对本实验所采用的**全 1 可分离卷积核、行主序打包方式以及先水平后垂直累加模型**得到的结构性下界。对于一般不可分离卷积核，旋转次数也可能高于 4 次。

七. 实验总结

本实验基于 Microsoft SEAL 4.1.1 开源全同态加密库，采用 CKKS 同态加密方案，实现了单输入单输出 4×4 输入矩阵与 3×3 卷积核的密文二维卷积计算，并在此基础上完成了密文旋转次数优化分析。实验内容对应课程实验五和实验六的要求，涵盖了密文卷积实现、正确性验证以及算法优化等多个方面。

实验首先完成了 CKKS 参数配置、密钥生成以及输入矩阵的 SIMD 打包，将整个 4×4 输入矩阵编码到单个密文中。随后，程序通过密文旋转、明文乘法以及密文累加等操作，实现了二维卷积计算。实验结果表明，密文卷积解密后的输出结果与普通明文卷积结果保持一致，两次独立运行的最大绝对误差均保持在 10^{-6} 数量级，满足实验精度要求，验证了程序实现的正确性。

在作业6中，我们进一步分析了“打包—旋转—累加”策略中的密文旋转次数。对于朴素实现，由于需要处理卷积核的 9 个位置，其中除原始密文外，其余 8 个偏移均需要执行一次密文旋转，因此共进行了 8 次 Rotation 操作。针对实验采用的全 1 卷积核具有可分离特性的特点，程序将二维卷积拆分为水平累加和垂直累加两个阶段，仅需分别完成两次密文旋转即可实现相同的卷积计算，从而将旋转次数降低至 4 次，较朴素算法减少了 50%，达到了本实验条件下的理论最优旋转次数。

通过本次实验，我们加深了对 CKKS 同态加密方案基本原理的理解，掌握了 Microsoft SEAL 的基本开发流程，熟悉了 CKKS 编码、SIMD 数据打包、密文旋转、密文与明文乘法、密文累加以及结果解密等关键技术。同时，对密文卷积的实现机制及其计算开销有了更加直观的认识，理解了密文旋转在同态卷积中的重要作用，以及合理设计数据布局和旋转策略对提升同态计算效率的重要意义。